Building Web and Mobile Apps WS2025/26 (Prof. Dr. Cathrin Möller)

Exercise 6 - 01.12./02.12.2025

Do not hesitate to ask questions or if you want feedback for your coding style. Unfortunately, we cannot comment in the GitLab directly.

Please commit and push to GitLab regularly and ensure @*cathrin.moeller* and my two tutors @pinky-jitendra.tinani and @rahul.patil are invited as a reporter.

Learning goals for today

Today we want to use the **HttpClient** and the **Forms** feature of Angular.
We want to read data from public assets (as a JSON file).
A **CRUD** application for users with addresses will be partially implemented.
The **Update/Edit** screen will be the most interesting as it shows the original persisted data which we can modify - using 2-way binding.
We will learn how to **map and cast different objects** as in Forms with 2-way binding, nested undefined accessors are not an option.
We also want to **connect** our **Angular** frontend **to** the **expressJS** backend.

Practical Exercises

This exercise is building up on exercise 5. If you did not do exercise 5 yet, start from there.

1) To finish your User(Detail) page implementation in the **UserComponent**, there are some actions left. Last week we connected the path param for the userId to the UserComponent. Now the goal is to **display correct data** for the user with that id.
Add a ***`getUser(userId: number)`*** method to the **UserService**. Call it from the UserComponent to get your full user information from the list of users.

Hint: You might need to **typecast** the path param (string if existing) to a number and use the result to call the service:

```
parseInt(this.userId)
```

Edit the template of the UserComponent to **show the user in a formatted way**. Test that **you can reach the user detail page** from the users screen.

The result should look similar to this:



2) Until now we have hardcoded users and addresses in a UserService.

a) Refactor that code to extract the users to a **JSON file** located in the **public** folder.

Info: In earlier Angular versions the *public* folder was named *assets*.

You will have to go back to a valid JSON format where also the key values have double quotes. Maybe you can copy the file from your vanilla or express JS code as long as the schema (attribute names) match.

b) Now add the **provideHttpClient**() dependency in the **providers** array to the app.config.ts (standalone) or your app.module.ts (non standalone). Now you can add the **HttpClient** as a dependency to the constructor of your UserService.

Add a method **fetchUsers**() to your **UserService** which uses the httpClient to get the data from ‚localhost:4200/users.json' (assuming your json file was placed in /public). Return that call to return an Observable. Call that fetchUsers() method in the **ngOnInit** of your UsersComponent and copy the subscribed result into a local „users" attribute of the component. If everything works right, you will be able to see the list of users again in your component.

Pitfalls: Maybe you need to parse the received result (users = data?.users). Logging the received data to the console will help.

Hint: If you are not familiar yet with RXJS, if you want to store a copy of users in your service, you can read and process the data received by subscribing to the observable before forwarding:

```
data.subscribe((data) => this.users = data.users); // forward data
```

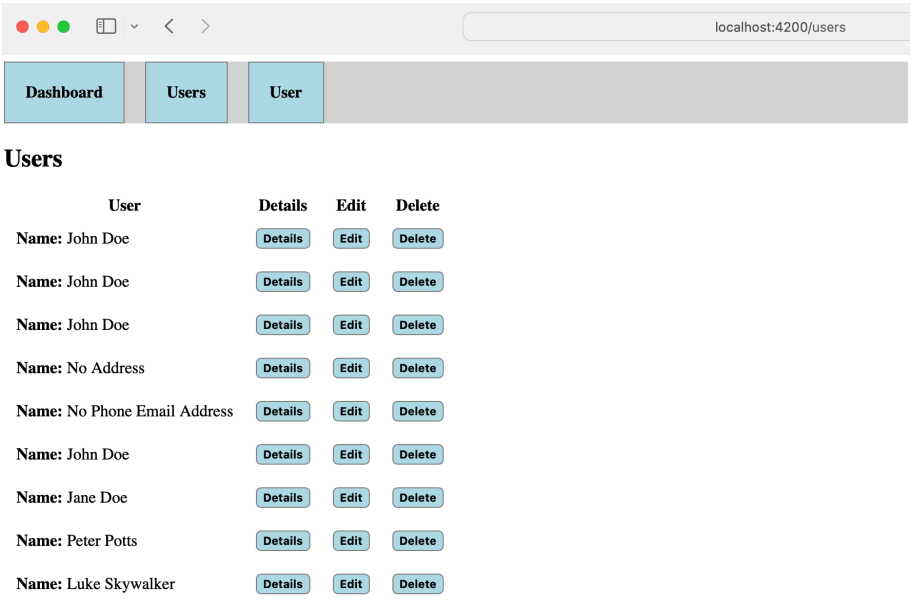
3) We want to create a simulated **CRUD** application. You should have two components that show data **read**-only for User(Detail) (by id) and the Users (List view).

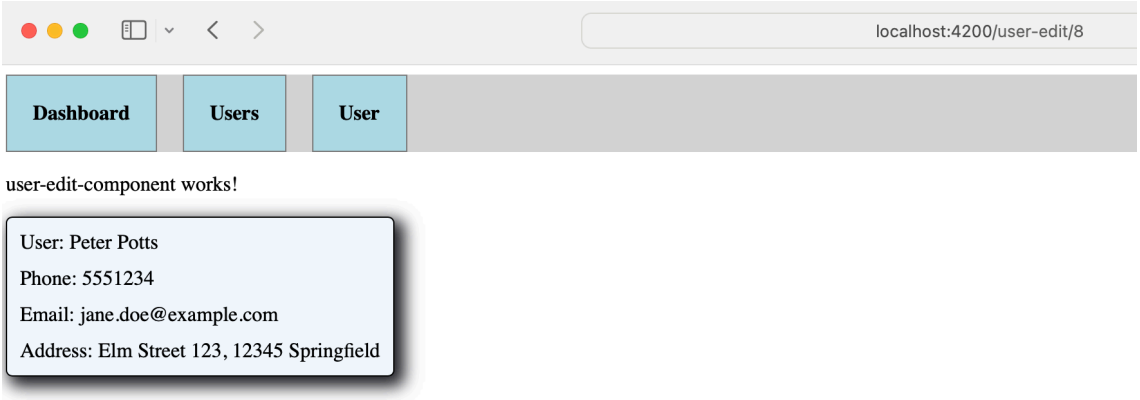
a) Create a **UserEditComponent** with a route having a path parameter for the id. Copy the logic from the User(Detail)Component to just display user data for now.

b) Add an **edit button** to each user entry in the UsersComponent template to reach the route (with id) that forwards to the UserEditComponent.

c) Add also a **delete button to each user entry in the list view of the UsersComponent**. It will not be functional yet.

Your user list and user edit screen (for now copy from read-only) should now look similar to this:

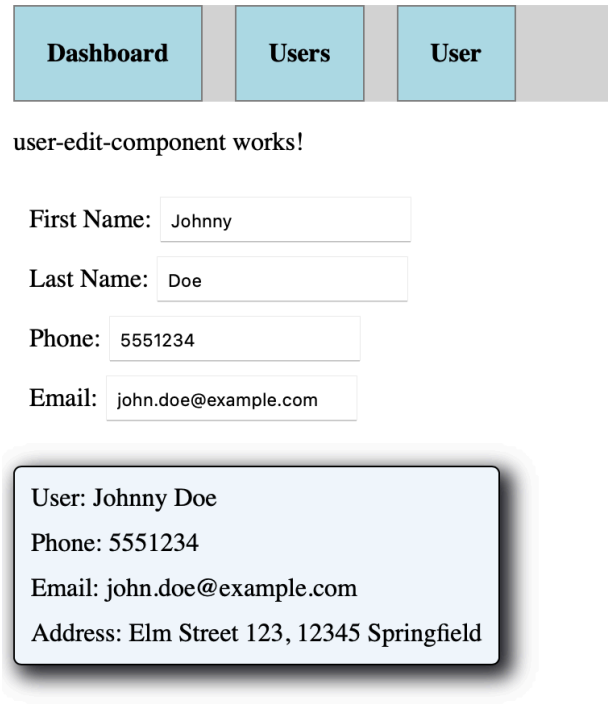




4) Now import the **FormsModule** to your UserEditComponent.

You can add a **template form** now to display the user data. Create one with a two way binding which displays the existing values inside the **input fields** and allows you to type and change. Skip the address for now. If you used optionals in typescript, this will cause issues. Add a stub for a submit function which does nothing for now. In future, we could implement a service method call which would trigger a POST API call to the expressJS backend.

The view should look similar to this (I kept a read-only display of user data at the bottom):



5) Now let's connect the Angular App to the **expressJS** backend from former exercises. We want to replace the http.get request in the **UserService** pointing not to our local JSON file but to the expressJS backend (running under localhost:8081).

Info: If you use a string constant for the backendAPI at the top of the service file, switching config for the URL is maintainable:

```
const getUsersAPI = 'http://localhost:8081/users'; //'http://localhost:4200/users.json';
```

We had a delay of 3s for the response implemented in exercise class 4. Reuse that endpoint (you can reduce the delay to 1s if you like).
Add a logic to the **UsersComponent** list view to show a **spinner** if the users are not loaded yet. Show the users and hide the spinner once received. You can use @if and @else for that in the template. You can copy paste the CSS from the React App in Exercise 4.

Hint: state management with asynchronous data is not trivial. If you have any logical issue in your code you might get stuck with the spinner.